Java Regex

What is Java Regex

- The **Java Regex** or Regular Expression is an API to *define a pattern for searching or manipulating strings*.
- It is widely used to define the constraint on strings such as password and email validation.
- Java Regex API provides 1 interface and 3 classes in **java.util.regex** package.

java.util.regex package

- The java.util.regex package provides following classes and interfaces for regular expressions.

1.MatchResult interface

2.Matcher class

3.Pattern class

4.PatternSyntaxException class

Pattern class

- It is the *compiled version of a regular expression*. It is used to define a pattern for the regex engine.
- *We can't create a Pattern object directly because it has no public constructor*
- *We use the static compile method eg:*

```
Pattern p = Pattern.compile(".ca");//. represents single character
```

Methods of Pattern class

No.	Method	Description
1	static Pattern compile(String regex)	compiles the given regex and returns the instance of the Pattern.
2	Matcher matcher(CharSequence input)	creates a matcher that matches the given input with the pattern.
3	static boolean matches(String regex, CharSequence input)	It works as the combination of compile and matcher methods. It compiles the regular expression and matches the given input with the pattern.
4	String[] split(CharSequence input)	splits the given input string around matches of given pattern.
5	String pattern()	returns the regex pattern.

Matcher class

- It implements the **MatchResult** interface. It is a *regex engine* which is used to perform match operations on a character sequence.
- We don't create the matcher directly either
- We use the matcher method of the pattern object to create a Matcher from the string we want to search in

```
Matcher m = p.matcher("rca");
```

Methods of Matcher class

No.	Method	Description
1	boolean matches()	test whether the regular expression matches the pattern.
2	boolean find()	finds the next expression that matches the pattern.
3	boolean find(int start)	finds the next expression that matches the pattern from the given start number.
4	String group()	returns the matched subsequence.
5	int start()	returns the starting index of the matched subsequence.
6	int end()	returns the ending index of the matched subsequence.
7	int groupCount()	returns the total number of the matched subsequence.

Example of Java Regular Expressions

```java
import java.util.regex.*;
public class MatchingMain {

    public static void main(String[] args) {
        //1st way
        Pattern p = Pattern.compile(".ca");//. represents single character
        Matcher m = p.matcher("rca");
        boolean b = m.matches();

        //2nd way
        boolean b2=Pattern.compile(".ca").matcher("rca").matches();

        //3rd way
        boolean b3 = Pattern.matches(".ca", "rca");

        System.out.println(b+" "+b2+" "+b3);

    }

}
```

Regex Character classes

No.	Character Class	Description
1	[abc]	a, b, or c (simple class)
2	[^abc]	Any character except a, b, or c (negation)
3	[a-zA-Z]	a through z or A through Z, inclusive (range)
4	[a-d[m-p]]	a through d, or m through p: [a-dm-p] (union)
5	[a-z&&[def]]	d, e, or f (intersection)
6	[a-z&&[^bc]]	a through z, except for b and c: [ad-z] (subtraction)
7	[a-z&&[^m-p]]	a through z, and not m through p: [a-lq-z](subtraction)

Regular Expression Character classes Example

```
import java.util.regex.*;
class RegexExample3{
public static void main(String args[]){
System.out.println(Pattern.matches("[amn]", "abcd"));//false (not a or m or n)
System.out.println(Pattern.matches("[amn]", "a"));//true (among a or m or n)
System.out.println(Pattern.matches("[amn]", "ammmna"));//false (m and a comes more than once)
}}
```

Regex Quantifiers

- The quantifiers specify the number of occurrences of a character.

Regex	Description
X?	X occurs once or not at all
X+	X occurs once or more times
X*	X occurs zero or more times
X{n}	X occurs n times only
X{n,}	X occurs n or more times
X{y,z}	X occurs at least y times but less than z times

Regular Expression Character classes and Quantifiers Example

```
System.out.println("? quantifier ....");
System.out.println(Pattern.matches("[amn]?", "a"));//true (a or m or n comes one time)
System.out.println(Pattern.matches("[amn]?", "aaa"));//false (a comes more than one time
System.out.println(Pattern.matches("[amn]?", "aammmnn"));//false (a m and n comes more t
System.out.println(Pattern.matches("[amn]?", "aazzta"));//false (a comes more than one t
System.out.println(Pattern.matches("[amn]?", "am"));//false (a or m or n must come one t

System.out.println("+ quantifier ....");
System.out.println(Pattern.matches("[amn]+", "a"));//true (a or m or n once or more time
System.out.println(Pattern.matches("[amn]+", "aaa"));//true (a comes more than one time)
System.out.println(Pattern.matches("[amn]+", "aammmnn"));//true (a or m or n comes more
System.out.println(Pattern.matches("[amn]+", "aazzta"));//false (z and t are not matchin

System.out.println("* quantifier ....");
System.out.println(Pattern.matches("[amn]*", "ammmna"));//true (a or m or n may come zer
```

Regex Metacharacters

- The regular expression metacharacters work as shortcodes.

Regex	Description
.	Any character (may or may not match terminator)
\d	Any digits, short of [0-9]
\D	Any non-digit, short for [^0-9]
\s	Any whitespace character, short for [\t\n\x0B\f\r]
\S	Any non-whitespace character, short for [^\s]
\w	Any word character, short for [a-zA-Z_0-9]
\W	Any non-word character, short for [^\w]
\b	A word boundary
\B	A non word boundary

Exercises

1. Write a Java program to validate email address using regular expression.
2. Write a Java program to validate Rwandan mobile telephone number using regular expression. The program should display message "Local number format" in case a number entered is starting with 07 or message "International number format" in case the entered number is starting with +2507
3. Write a Java program to validate strong password using regular expression. A strong password should contain at least 8 characters including alphanumeric characters, special characters and at least one character should be capital character.